Computer System Architecture
Lecture 1
Chapter 3: Data Representation

Based on M. Morris Mano, Computer System Architecture, 3rd ed.

1 The Central Problem: What Do Bits Mean?

A digital computer ultimately stores information in registers and memory as binary values. A register consists of binary storage elements, so physically it contains only patterns of 0's and 1's.

Those bit patterns may represent very different kinds of information.

They may represent:

- numbers used in arithmetic,
- letters and other characters,
- discrete symbols,
- or control information used to direct computer operations.

The bits alone therefore do not tell us what the stored information means. The meaning depends upon the *representation* being used.

WHITEBOARD

DATA REPRESENTATION

Registers store bits: $0, 1$

Bits may represent numbers, characters, or other symbols.

$$\boxed{\text{bit pattern} + \text{interpretation} = \text{meaning}}$$

This idea will run throughout the chapter. The same physical bit pattern can be interpreted differently depending upon what type of data the computer expects it to represent.

2 Number Systems and Radix

A positional number system has a *base*, also called a *radix*, denoted by r.

A number written in radix r consists of digits whose positions are weighted by integer powers of r. In general,

$$N = \sum_i d_i r^i,$$

where d_i is the digit occupying position i.

The four number systems of greatest interest here are:

Number System	Radix
Binary	2
Octal	8
Decimal	10
Hexadecimal	16

In hexadecimal, the symbols

$$A, B, C, D, E, F$$

represent decimal values

$$10, 11, 12, 13, 14, 15.$$

WHITEBOARD

Radix r

$$N = \sum_i d_i r^i$$

System	r
binary	2
octal	8
decimal	10
hexadecimal	16

$$A = 10, \quad B = 11, \quad \ldots, \quad F = 15$$

For example,

$$(101101)_2$$

has the decimal value

$$1(2^5) + 0(2^4) + 1(2^3) + 1(2^2) + 0(2^1) + 1(2^0).$$

Therefore,

$$(101101)_2 = (45)_{10}.$$

WHITEBOARD

$$(101101)_2$$

$$= 1(2^5) + 0(2^4) + 1(2^3) + 1(2^2) + 0(2) + 1$$

$$= 32 + 8 + 4 + 1$$

$$\boxed{(101101)_2 = (45)_{10}}$$

---

3 Converting Decimal Integers to Another Radix

To convert a decimal integer into radix r:

1. Divide the number by r.
2. Record the remainder.
3. Divide the quotient by r again.
4. Continue until the quotient is zero.
5. Read the remainders from last to first.

Consider the conversion of 13_{10} to binary.

WHITEBOARD

$\boxed{\text{Decimal integer} \rightarrow \text{base } r}$

Repeatedly divide by r.

Read the remainders upward.

$$\begin{aligned} 13/2 &= 6 \quad R1 \\ 6/2 &= 3 \quad R0 \\ 3/2 &= 1 \quad R1 \\ 1/2 &= 0 \quad R1 \end{aligned}$$

Reading upward:

$$\boxed{(13)_{10} = (1101)_2}$$

4 Converting Decimal Fractions

The fractional part of a decimal number is converted differently.

To convert a decimal fraction into radix r:

1. Multiply the fraction by r.
2. Record the integer part of the result.
3. Multiply the remaining fractional part by r.
4. Continue until the fraction becomes zero or the desired precision has been obtained.
5. Read the generated integer digits from first to last.

For example, convert

$$(0.625)_{10}$$

to binary.

WHITEBOARD

$$\boxed{\text{Decimal fraction} \to \text{base } r}$$

Repeatedly multiply by r.

Read integer parts downward.

$$0.625(2) = 1.25 \Rightarrow 1$$

$$0.25(2) = 0.50 \Rightarrow 0$$

$$0.50(2) = 1.00 \Rightarrow 1$$

$$\boxed{(0.625)_{10} = (0.101)_2}$$

For a mixed number, the integer and fractional portions are converted separately and then joined at the radix point.

5 Binary, Octal, and Hexadecimal

Octal and hexadecimal are especially convenient when working with binary numbers because their radices are powers of two:

$$8 = 2^3, \qquad 16 = 2^4.$$

Therefore,

$$\boxed{1 \text{ octal digit} = 3 \text{ binary bits}}$$

and

$$\boxed{1 \text{ hexadecimal digit} = 4 \text{ binary bits}}.$$

This makes conversion between these systems a matter of grouping bits rather than performing arithmetic.

WHITEBOARD

$$8 = 2^3 \Rightarrow 1 \text{ octal digit} = 3 \text{ bits}$$

$$16 = 2^4 \Rightarrow 1 \text{ hex digit} = 4 \text{ bits}$$

For hexadecimal:

$$1010 \quad 1111 \quad 0110 \quad 0011$$

$$A \qquad F \qquad 6 \qquad 3$$

$$\boxed{(1010111101100011)_2 = (AF63)_{16}}$$

For octal:

$$\boxed{\text{Group binary bits in groups of 3.}}$$

Octal and hexadecimal are convenient ways of displaying the contents of binary registers because they greatly shorten long strings of 0's and 1's.

6 Binary Numbers Versus Binary Codes

A crucial distinction must now be made.

A number can be *converted* into binary, or its decimal digits can be *encoded* with binary patterns. These are not the same operation.

For example,

$$(99)_{10} = (1100011)_2.$$

This is the ordinary binary representation of the numerical quantity ninety-nine.

In binary-coded decimal, or BCD, each decimal digit is instead encoded separately using four bits:

$$9 \rightarrow 1001.$$

Therefore,

$$99_{\text{BCD}} = 1001\ 1001.$$

WHITEBOARD

Binary number $\neq$ binary-coded decimal

$$99_{10} = 1100011_2$$

but

$$99_{\text{BCD}} = 1001\ 1001$$

BCD encodes each decimal digit separately.

The BCD assignments are

Decimal Digit	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Because four bits permit $2^4 = 16$ different patterns but only ten decimal digits exist, six BCD patterns are unused.

WHITEBOARD

$$0 \rightarrow 0000, \quad \ldots, \quad 9 \rightarrow 1001$$

1010 through 1111 are unused in BCD.

---

7 Alphanumeric Representation and ASCII

Computers must also represent letters, digits, punctuation marks, and special characters.

An *alphanumeric code* assigns a binary pattern to each character in a character set.

The standard code discussed by Mano is ASCII, the American Standard Code for Information Interchange.

ASCII uses seven bits, permitting

$$2^7 = 128$$

different character codes.

Examples include

$$A = 1000001$$

and

$$0 = 0110000.$$

WHITEBOARD

$$\boxed{\text{ASCII}}$$

$$7 \text{ bits} \Rightarrow 2^7 = 128 \text{ possible codes}$$

$$A = 1000001$$

$$0 = 0110000, \qquad 9 = 0111001$$

An interesting relationship exists between the ASCII codes for decimal digits and BCD. The ASCII digits begin with the three high-order bits 011.

Thus,

$$011\,0000$$

is ASCII 0, while

$$011\,1001$$

is ASCII 9.

Removing the high-order 011 leaves the corresponding four-bit BCD digit.

WHITEBOARD

ASCII decimal digit: $011\,xxxx$

Remove 011

$\Downarrow$

$\boxed{\text{BCD digit } xxxx}$

8 Complements

Complements are useful because they allow subtraction to be performed through addition.

For a base-r, n-digit number N, Mano defines two complements.

The $(r-1)$'s complement is

$$(r^n - 1) - N.$$

The r's complement is

$$r^n - N$$

for nonzero N.

WHITEBOARD

$\boxed{(r-1)\text{'s complement} = (r^n - 1) - N}$

$\boxed{r\text{'s complement} = r^n - N}$

	$r-1$	r
decimal	9's	10's
binary	1's	2's

8.1 1's Complement

The 1's complement of a binary number is obtained by changing every

$$0 \to 1$$

and every

$$1 \to 0.$$

For example,

$$1011001$$

becomes

$$0100110.$$

WHITEBOARD

1's complement: flip every bit

$$1011001$$

$$\Downarrow$$

$$0100110$$

8.2 2's Complement

The 2's complement is obtained by first taking the 1's complement and then adding one.

For example,

$$101100$$

has 1's complement

$$010011.$$

Adding one gives

$$010011 + 1 = 010100.$$

WHITEBOARD

2's complement: flip every bit, then add 1

101100

$\Downarrow$

010011

+1

010100

A useful shortcut is to begin at the right side of the number.

Leave all trailing 0's and the first 1 unchanged. Complement every remaining bit to the left.

WHITEBOARD

2's-complement shortcut:
from the right, keep trailing zeros and the first 1;
flip everything to the left.

---

9 Subtraction Using Complements

Suppose two n-bit unsigned numbers M and N are to be subtracted.

Instead of directly computing

$$M - N,$$

we may add the 2's complement of N:

$$M + (2^n - N).$$

Algebraically,

$$M + (2^n - N) = M - N + 2^n.$$

If an end carry occurs, the 2^n term appears as that carry and is discarded.

WHITEBOARD

$$\boxed{M - N = M + (2^n - N)}$$

$$M + (2^n - N) = M - N + 2^n$$

Subtraction can be implemented with addition hardware.

Consider $7 - 5$ using four bits.

$$7 = 0111, \qquad 5 = 0101.$$

The 2's complement of 0101 is

$$1011.$$

Therefore,

$$0111 + 1011 = 1\,0010.$$

The end carry is discarded, leaving

$$0010 = 2.$$

WHITEBOARD

$$7 - 5$$

$$7 = 0111, \qquad 5 = 0101$$

$$\text{2's comp. of } 0101 = 1011$$

$$\begin{array}{r} 0111 \\ +1011 \\ \hline 1\,0010 \end{array}$$

discard carry

$$\boxed{0010 = 2}$$

10 Fixed-Point Representation

A binary number may represent an integer, a fraction, or a mixed quantity.

The binary point itself does not ordinarily need to occupy a physical storage position. Instead, its location can be assumed.

In *fixed-point representation*, the binary point always occupies an assumed fixed position.

Two common arrangements are:

- binary point assumed at the extreme right, producing integers;
- binary point assumed at the extreme left, producing fractions.

WHITEBOARD

Fixed point: the radix-point position is assumed.

point at right ⇒ integer

point at left ⇒ fraction

11 Signed Integer Representation

A sign must also be represented with a bit.

The convention used is

$$0 = \text{positive}, \qquad 1 = \text{negative}.$$

For negative integers, Mano discusses three representations:

1. signed magnitude,
2. signed 1's complement,
3. signed 2's complement.

Consider 14 in an 8-bit register.

$$+14 = 00001110.$$

The three representations of -14 are:

Representation	-14
Signed magnitude	10001110
Signed 1's complement	11110001
Signed 2's complement	11110010

WHITEBOARD

$$0 = \text{positive sign}, \qquad 1 = \text{negative sign}$$

$$+14 = 00001110$$

Representation of -14	Bits
signed magnitude	10001110
1's complement	11110001
2's complement	11110010

Signed magnitude resembles ordinary written arithmetic but makes computer arithmetic comparatively awkward.

The 1's-complement representation also has the inconvenience of possessing two representations of zero:

$$+0 \qquad \text{and} \qquad -0.$$

For this reason, Mano's discussion of signed binary arithmetic concentrates on the 2's-complement system.

WHITEBOARD

---

signed magnitude: awkward arithmetic

1's complement: $+0$ and -0

2's complement is preferred for signed arithmetic.

---

12 Signed 2's-Complement Addition

The great advantage of 2's complement is that the arithmetic rule is extremely simple:

1. Add both numbers, including their sign bits.
2. Discard any carry out of the sign-bit position.

Negative results remain naturally expressed in 2's-complement form.

WHITEBOARD

2's-complement addition

Add all bits, including sign bits.

Discard carry out of the sign bit.

Consider

$$(+6) + (-13).$$

In eight bits,

$$+6 = 00000110.$$

To represent -13,

$$13 = 00001101,$$

so

$$-13 = 11110011.$$

Therefore,

$$\begin{array}{r} 00000110 \\ +11110011 \\ \hline 11111001 \end{array}$$

The result has sign bit 1, so it is negative.

Taking its 2's complement,

$$11111001 \rightarrow 00000111.$$

Thus,

$$11111001 = -7.$$

WHITEBOARD

$$+6 = 00000110$$

$$-13 = 11110011$$

$$\begin{array}{r} 00000110 \\ +11110011 \\ \hline 11111001 \end{array}$$

$$11111001 \xrightarrow{\text{2's comp.}} 00000111$$

$$\boxed{11111001 = -7}$$

---

13 Signed 2's-Complement Subtraction

Subtraction follows directly from

$$A - B = A + (-B).$$

Therefore, to subtract B from A:

1. Take the 2's complement of B.
2. Add it to A.
3. Discard any carry out of the sign position.

WHITEBOARD

$$\boxed{A - B = A + (-B)}$$

2's-complement the subtrahend, then add.

This means that the same basic binary addition circuitry can be used for both addition and subtraction.

14 Overflow

A computer register has finite width. If the true arithmetic result cannot fit within that width, an *overflow* occurs.

The interpretation of overflow depends upon whether the operands are unsigned or signed.

14.1 Unsigned Overflow

For unsigned numbers, overflow is indicated by a carry out of the most significant bit.

WHITEBOARD

Unsigned overflow: carry out of the MSB

14.2 Signed 2's-Complement Overflow

For signed 2's-complement numbers, the final carry alone does not determine overflow.

Overflow can occur when two operands have the same sign but the stored result has the opposite sign.

Thus:

$$(+) + (+) \rightarrow (-)$$

indicates overflow, and

$$(-) + (-) \rightarrow (+)$$

also indicates overflow.

WHITEBOARD

---

same-sign operands + opposite-sign result ⇒ overflow

Mano gives a hardware test based upon the carry into and carry out of the sign bit.

Let

$$C_{\text{in}}$$

be the carry into the sign position and

$$C_{\text{out}}$$

be the carry out of it.

Then

$$V = C_{\text{in}} \oplus C_{\text{out}}.$$

Therefore,

$$C_{\text{in}} \neq C_{\text{out}}$$

indicates overflow.

WHITEBOARD

$$\boxed{V = C_{\text{in sign}} \oplus C_{\text{out sign}}}$$

$$C_{\text{in}} \neq C_{\text{out}} \Rightarrow V = 1$$

$$\boxed{\text{Overflow}}$$

An n-bit 2's-complement register represents values in the range

$$-2^{n-1} \leq x \leq 2^{n-1} - 1.$$

For eight bits,

$$-128 \leq x \leq 127.$$

Thus, although

$$70 + 80 = 150,$$

the value 150 cannot be represented as an 8-bit signed 2's-complement integer.

WHITEBOARD

$$\boxed{-2^{n-1} \leq x \leq 2^{n-1} - 1}$$

For $n = 8$:

$$-128 \leq x \leq 127$$

$$70 + 80 = 150 > 127$$

$$\boxed{\text{overflow}}$$

15 Decimal Fixed-Point Representation

Decimal values can also be stored and manipulated directly in a binary-coded form.

Using BCD requires four bits for every decimal digit.

For example,

4385

is represented as

0100 0011 1000 0101.

WHITEBOARD

Decimal fixed point uses coded decimal digits.

$4385_{\text{BCD}} = 0100\ 0011\ 1000\ 0101$

4 bits per decimal digit

Decimal representation requires more storage than the equivalent pure binary number, and decimal arithmetic circuitry is more complicated.

Its advantage is that decimal-oriented input, output, and processing can be performed without repeatedly converting between decimal and binary forms.

WHITEBOARD

BCD: more storage

BCD arithmetic: more complex hardware

advantage: convenient for decimal I/O and processing

The decimal counterpart of binary 2's-complement arithmetic is the 10's-complement system.

WHITEBOARD

Decimal analogue of 2's complement: 10's complement

---

16 Floating-Point Representation

Fixed-point representation assumes that the radix point remains in one fixed position.

Floating-point representation instead stores information that allows the effective radix-point position to vary.

Mano expresses a floating-point quantity as

$$m \times r^e,$$

where

$$m = \text{mantissa}$$

and

$$e = \text{exponent}.$$

The radix r and the location of the radix point within the mantissa are understood rather than explicitly stored.

WHITEBOARD

$$\boxed{\text{Floating point} = m \times r^e}$$

$$m = \text{mantissa}$$

$$e = \text{exponent}$$

For example,

$$6132.789 = 0.6132789 \times 10^4.$$

WHITEBOARD

$$6132.789 = 0.6132789 \times 10^4$$

The same principle applies in binary.

For example,

$$1001.11_2 = 0.100111_2 \times 2^4.$$

WHITEBOARD

$$1001.11_2 = 0.100111_2 \times 2^4$$

16.1 Normalization

A floating-point number is *normalized* when the most significant digit of the mantissa is nonzero.

Normalization moves the radix point while changing the exponent by a corresponding amount so that the numerical value remains unchanged.

For example,

$$0.001101 \times 2^5$$

may be normalized by shifting the mantissa two positions left:

$$0.1101 \times 2^3.$$

The value has not changed.

WHITEBOARD

normalized: most significant mantissa digit is nonzero

$$0.001101 \times 2^5$$

shift left two places:

$$0.1101 \times 2^3$$

same value

Normalization maximizes available precision.

Floating-point arithmetic requires more complicated hardware than fixed-point arithmetic, but it permits a much wider range of numerical magnitudes.

17 Gray Code

The reflected binary code, usually called *Gray code*, has the important property that adjacent values differ in only one bit.

A portion of the Gray-code sequence is

$$000,\ 001,\ 011,\ 010,\ 110,\ 111, \ldots$$

WHITEBOARD

Gray code: adjacent values differ by one bit.

$$000 \to 001 \to 011 \to 010$$

$$\to 110 \to 111 \to \cdots$$

Only one bit changes at each step.

This property is useful when a physical or digital state passes from one value to the next because it reduces ambiguity during the transition.

WHITEBOARD

Advantage: reduced ambiguity during transitions

---

18 Other Decimal Codes

BCD is not the only possible binary code for decimal digits.

Mano also discusses:

- 8421 BCD,

- 2421 code,
- Excess-3 code,
- Excess-3 Gray code.

BCD may be viewed as an 8421 weighted code because the bit positions have weights

$$8,\ 4,\ 2,\ 1.$$

The 2421 code is another weighted code.

The Excess-3 code is obtained by adding binary 3, or 0011, to the BCD representation of each decimal digit.

The 2421 and Excess-3 codes have a self-complementing property: complementing all bits produces the code representation of the 9's complement of the decimal digit.

WHITEBOARD

Other decimal codes

BCD = 8421 weighted code

2421: weighted and self-complementing

Excess-3 = BCD + 0011

2421 and Excess-3: bit complement ⇒ 9's complement

---

19 Other Alphanumeric Codes

Mano also identifies other character-code arrangements.

ASCII uses seven bits for each character and commonly receives an additional parity bit during transmission.

EBCDIC, associated with IBM equipment, uses eight bits per character.

Some systems may also use shorter internal character codes when a smaller character set is sufficient.

WHITEBOARD

Alphanumeric codes

ASCII: 7-bit character code

EBCDIC: 8-bit character code

smaller internal codes may also be used

20 Error Detection and Parity

Binary information sent through a communication medium may be affected by noise.

A transmitted

1

may become

0,

or a transmitted

0

may become

1.

An error-detection code adds information that allows the receiver to recognize that some transmission errors have occurred.

The most common method discussed by Mano is the *parity bit*.

WHITEBOARD

Transmission error: $1 \leftrightarrow 0$

Parity bit: extra bit used for error detection

20.1 Even and Odd Parity

With *even parity*, the parity bit is chosen so that the total number of 1's in the transmitted word is even.

With *odd parity*, the parity bit is chosen so that the total number of 1's is odd.

WHITEBOARD

Even parity: total number of 1s is even

Odd parity: total number of 1s is odd

Suppose the message is

$$1011.$$

It contains three 1's.

To obtain even parity, the parity bit must therefore be

$$P = 1,$$

giving four 1's in the transmitted word.

WHITEBOARD

Message:

$$1011$$

Number of 1's:

$$3$$

For even parity:

$$P = 1$$

Total number of 1's:

$$4$$

$$\boxed{\text{even parity satisfied}}$$

20.2 Parity and Exclusive-OR

Parity generation connects directly to the exclusive-OR operation studied in digital logic.

For a three-bit message x, y, z,

$$P_{\text{even}} = x \oplus y \oplus z.$$

The exclusive-OR function is 1 whenever an odd number of its inputs are 1.

Odd parity therefore requires the complement:

$$P_{\text{odd}} = \overline{x \oplus y \oplus z}.$$

WHITEBOARD

$$\boxed{P_{\text{even}} = x \oplus y \oplus z}$$

$$\boxed{P_{\text{odd}} = \overline{x \oplus y \oplus z}}$$

XOR is an odd function.

20.3 Parity Generator and Checker

At the sending end, a parity generator calculates the required parity bit.

The message and parity bit are then transmitted.

At the receiving end, a parity checker determines whether the received bit pattern still has the required parity.

WHITEBOARD

---

message → [parity generator] → channel → [parity checker] → error indication

---

20.4 What Parity Can and Cannot Detect

A single parity bit detects an odd number of changed bits.

Thus it detects:

$$1,\ 3,\ 5, \ldots$$

bit errors.

However, if an even number of bits change, the parity may remain correct and the error can go undetected.

WHITEBOARD

[Parity detects an odd number of bit errors.]

$$1,\ 3,\ 5, \ldots$$

[An even number of bit errors may go undetected.]

Parity therefore provides *error detection*, not complete error protection and not automatic error correction.

21 Chapter 3 Summary

The fundamental theme of this chapter is that computers physically store binary information, but those bits acquire meaning only through an agreed representation.

We have seen that bits can represent:

- binary, octal, decimal, and hexadecimal quantities,
- decimal digits through BCD,
- characters through codes such as ASCII,
- positive and negative integers,
- fixed-point and floating-point quantities,
- alternate codes such as Gray code,
- and redundant information used for error detection.

The particularly important architectural consequence is that a representation is chosen not merely for human readability but also because it can simplify the hardware required to manipulate the information.

Two's-complement representation is a prime example: subtraction can be reduced to addition, allowing common arithmetic hardware to serve both operations.

WHITEBOARD






